SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai 2026

ASSESSMENT TOOL -2

C04_AT2

Name : S Nikhileswar Reddy

Register No : 192525096

Course code : CSA1613

Course name: Data Warehousing and Data Mining

Faculty name: Dr.Kanimozhi KV

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Data Warehousing and Data Mining COURSE CODE: CSA1613

COURSE OUTCOME (CO4): Examine the applicability of clustering algorithms in real-world scenarios, presenting findings through clear reports with effective visualizations.

Questions: 05 Total Marks: 50

Time Duration: 45 Minutes

Q1. Customer Data — K-Means vs Hierarchical Clustering

Dataset:

Customer	Income (₹)	Spending Score
C1	30000	60
C2	60000	40
C3	50000	70
C4	28000	65
C5	65000	35

Question: Compare K-means and hierarchical clustering using this dataset and explain differences in performance and scalability.

Answer:

Applying both methods to the 5-customer income/spending dataset shows two natural groups: a lower-income, higher-spending group (C1, C3, C4) and a higher-income, lower-spending group (C2, C5). K-means (with k = 2) assigns each point to the nearest of two randomly-initialized centroids and

iteratively updates centroid positions until convergence, quickly separating the customers into these two clusters based on Euclidean distance in the income–spending space. Hierarchical (agglomerative) clustering instead starts with each customer as its own cluster and successively merges the closest pairs (e.g., C1 with C4, C2 with C5) — producing a dendrogram that shows the same two-group split but also reveals the merge order and relative distances.

Aspect	**K-Means**	**Hierarchical Clustering**
Number of clusters	Must be specified in advance (k = 2 here)	Not required upfront; clusters chosen by cutting the dendrogram
Result on this data	Fast convergence to {C1,C3,C4} and {C2,C5}	Same grouping, but with a visual merge hierarchy
Time complexity	$O(n \cdot k \cdot i)$ — near-linear, very fast for small n=5	$O(n^2 \log n)$ or $O(n^3)$ depending on method — costlier
Scalability	Scales well to large datasets (thousands+ points)	Becomes impractical for large n due to distance-matrix storage/computation
Sensitivity	Sensitive to initial centroid choice and outliers	Sensitive to the linkage criterion (single/complete/average)
Interpretability	Only final flat partition is visible	Dendrogram gives richer structural insight

Conclusion: For this small dataset both algorithms converge to the same two clusters, but K-means is more efficient and scalable for larger customer bases, while hierarchical clustering is preferable when the number of clusters is unknown or when the merge structure itself is informative.

Q2. Document Data — K-Means vs EM for Overlapping Clusters

Dataset:

Document	**Word1**	**Word2**	**Word3**
D1	0.2	0.5	0.3
D2	0.1	0.6	0.3

Document	Word1	Word2	Word3
D3	0.8	0.1	0.1
D4	0.75	0.15	0.1
D5	0.2	0.4	0.4

Question: Compare K-means and EM algorithms in handling overlapping clusters for the given dataset.

Answer:

In this term-weight dataset, D3 and D4 form a clearly separated, dense cluster (high Word1, low Word2/Word3), while D1, D2 and D5 are close together but not identical — D5 in particular sits between the D1/D2 pair and could plausibly belong to either group, illustrating overlap. K-means assigns every document to exactly one cluster based on nearest centroid (hard assignment), so it would force D5 into a single cluster with D1/D2 even though its Word3 value (0.4) is noticeably higher, discarding the partial similarity. The Expectation-Maximization (EM) algorithm, based on a Gaussian Mixture Model, instead computes a probability of membership for each document in each cluster (soft assignment) — so D5 could be reported as, say, 65% likely in the D1/D2 cluster and 35% likely in a boundary/overlap region, capturing the ambiguity that hard clustering hides.

Aspect	K-Means	EM (Gaussian Mixture Model)
Assignment type	Hard — each document belongs to exactly one cluster	Soft — probabilistic membership across clusters
Overlap handling	Forces a single label even for borderline points like D5	Naturally expresses D5's partial membership in multiple clusters
Cluster shape assumption	Assumes spherical, equal-sized clusters	Can model elliptical clusters of varying size/orientation via covariance
Computation	Simple distance minimization, fast to converge	Iterative log-likelihood maximization, more computationally intensive

Aspect	K-Means	EM (Gaussian Mixture Model)
Best suited when	Clusters are compact and well separated (e.g., D3–D4)	Clusters overlap or have varying density (e.g., D1–D2–D5 region)

Conclusion: K-means correctly separates the well-isolated D3/D4 pair but oversimplifies the D1/D2/D5 region; EM better represents this dataset's genuine overlap by assigning graded probabilities instead of a rigid boundary.

Q3. Geographic Data — Euclidean vs Manhattan Distance

Dataset:

Point	Latitude	Longitude
P1	13.08	80.27
P2	13.10	80.25
P3	12.97	80.22
P4	13.05	80.30
P5	13.00	80.20

Question: Compare Euclidean and Manhattan distance metrics and analyze their impact on clustering results.

Answer:

Euclidean distance measures the straight-line ('as the crow flies') distance between two coordinate points: $d = \sqrt{((lat1-lat2)^2 + (lon1-lon2)^2)}$. Manhattan distance measures the sum of absolute differences along each axis: $d = |lat1-lat2| + |lon1-lon2|$, resembling travel along a grid of city blocks. For example, between P1 (13.08, 80.27) and P4 (13.05, 80.30): the Euclidean distance is $\sqrt{((0.03)^2 + (0.03)^2)} \approx 0.0424$, whereas the Manhattan distance is $|0.03| + |0.03| = 0.06$ — Manhattan distance is always equal to or larger than Euclidean distance for the same pair of points.

Aspect	Euclidean Distance	Manhattan Distance
Formula	$\sqrt{(\Delta lat^2 + \Delta lon^2)}$	$\lvert\Delta lat\rvert + \lvert\Delta lon\rvert$

Aspect	Euclidean Distance	Manhattan Distance
Geometric meaning	Straight-line (direct) distance	Grid-based / axis-aligned travel distance
Sensitivity	More sensitive to large single-axis differences (squaring effect)	Treats each axis contribution linearly, more robust to outlying single-axis shifts
Effect on clustering	Tends to produce round/spherical clusters	Tends to produce diamond-shaped cluster boundaries
Suitability here	Suitable if geographic proximity is truly straight-line (e.g., aerial distance)	Suitable if points represent locations connected by a road/street grid

Conclusion: On this closely-spaced set of five points, both metrics would group P1, P2 and P4 together (they are mutually close) and place P3 and P5 as a southern pair or as more distant outliers, but the exact cluster boundaries and which algorithm (e.g., K-means using Euclidean vs. K-medoids using Manhattan) is appropriate depends on whether straight-line or grid-based proximity best reflects the real-world relationship between the points.

Q4. Retail Products — Agglomerative vs Divisive Clustering

Dataset:

Product	Sales (₹)	Frequency
P1	50000	20
P2	30000	10
P3	52000	22
P4	25000	8
P5	32000	12

Question: Compare agglomerative and divisive clustering approaches and explain their suitability for this dataset.

Model Answer:

The dataset naturally splits into a high-sales/high-frequency group (P1, P3) and a low-sales/low-frequency group (P2, P4, P5). Agglomerative clustering builds this result bottom-up: it starts with each of the five products as its own cluster and repeatedly merges the two nearest clusters — first merging P1 with P3 (very similar) and P4 with P5 or P2 (also close), and eventually combining these into two final groups. Divisive clustering works top-down: it starts with all five products in one cluster and recursively splits the least cohesive cluster — the first split would separate the {P1,P3} high-value pair from {P2,P4,P5}, and further splits could subdivide {P2,P4,P5} if a finer partition were needed.

Aspect	**Agglomerative (Bottom-Up)**	**Divisive (Top-Down)**
Starting point	Each product is its own cluster	All products start in a single cluster
Process	Iteratively merges the closest pairs/clusters	Iteratively splits the cluster with the highest internal dissimilarity
Computational cost	$O(n^2 \log n)$ typically — lower cost, widely used	$O(2^n)$ in the worst case — computationally expensive for exhaustive search
Result on this data	Efficiently merges P1+P3 and P2+P4(+P5)	Correctly isolates the high-value pair first, but costlier to compute for larger catalogs
Suitability	Preferred for this small, well-separated 5-product dataset — simple and efficient	Better when a small number of top-level segments (e.g., 'premium' vs 'budget') is the primary goal and dataset is small

Conclusion: For this dataset, agglomerative clustering is more practical since it is computationally cheaper and produces the same two natural product segments; divisive clustering would offer more direct control over the very first, most important split (premium vs. budget) but at greater computational expense as the retail catalog scales.

Q5. Mixed Attributes — Centroid-Based vs Density-Based Clustering

Dataset:

Item	Price (₹)	Category	Rating
I1	500	A	4.5
I2	300	B	3.8
I3	550	A	4.7
I4	250	B	3.5
I5	520	A	4.6

Question: Compare centroid-based and density-based clustering techniques and analyze their effectiveness on mixed data types.

Answer:

This dataset mixes a numeric attribute (Price, Rating) with a categorical attribute (Category). Centroid-based clustering (e.g., K-means) computes a mean centroid per cluster, which works naturally on Price and Rating but has no direct notion of an 'average' for the categorical Category field — it would typically require the category to be encoded numerically (e.g., one-hot encoding) or a variant such as K-prototypes to be used instead. Applied to Price and Rating alone, centroid-based clustering would correctly group I1, I3, I5 (high price/rating, Category A) separately from I2, I4 (low price/rating, Category B). Density-based clustering (e.g., DBSCAN) instead groups points that are closely packed together (within an ε-neighborhood with a minimum number of points) and labels sparse points as noise/outliers — it does not need a predefined number of clusters and can detect the same two dense groups (I1/I3/I5 and I2/I4) without assuming spherical shape, but it likewise needs a distance measure that meaningfully incorporates the categorical Category attribute.

Aspect	Centroid-Based (K-Means)	Density-Based (DBSCAN)
Cluster definition	Points closest to a computed mean centroid	Points in dense, connected neighborhoods (ε, minPts)
Handling numeric data	Directly computes mean of Price/Rating — straightforward	Uses distance thresholds on Price/Rating — straightforward

Aspect	Centroid-Based (K-Means)	Density-Based (DBSCAN)
Handling categorical data (Category)	No natural mean for categories; needs encoding or K-prototypes	Needs a mixed-distance metric (e.g., Gower distance) for categories
Cluster shape	Assumes roughly spherical clusters	Can find arbitrarily shaped, non-spherical clusters
Outlier handling	No explicit outlier detection — every point is assigned	Explicitly labels sparse points as noise/outliers
Predefined k	Required	Not required

Conclusion: On this mixed-attribute dataset, both techniques identify the same two intuitive groups (I1/I3/I5 vs. I2/I4) when applied to the numeric attributes, but neither handles the categorical Category field natively — centroid-based methods need encoding or a K-prototypes extension, while density-based methods need a mixed-type distance measure such as Gower distance; density-based clustering additionally offers the advantage of flagging genuine outliers rather than forcing every item into a cluster.